Lab 9: Containerising the Proximity Search API

Rahul Raj (12341680)

October 24, 2026

Project containerised	Lab 7, Proximity Search API (FastAPI, road-graph BFS)
Code	`github.com/Rahul5977/CSD-Course`, folder `Lab-Task/9.Docker`
Run it	`cd Lab-Task/9.Docker && docker compose up -d -build`
Open	`http://localhost:8088/` (API: `/search/?lat=&long=&cat=&rad=`)
Machine	personal MacBook (Apple silicon, macOS 26.6), Docker Desktop 29.1.3, Compose v5.0.1

1 What was built

In Lab 7 the API was one Python process that served both the JSON routes and the demo map page. For this lab it was split into services with one Dockerfile each, and one `docker-compose.yml` builds and starts all of them:

- **api**: the Lab 7 FastAPI application under uvicorn. It loads the 10,000 locations and 14,800 roads at start-up and answers `/search/`, `/health` and `/network`. It can run as several replicas.
- **web**: nginx. It serves the map page (static HTML) and is a reverse proxy and load balancer for the API routes. It is the only container with a port on the host.
- **loadtest**: the Lab 7 load-test script in its own image, run on demand (`profile: test`) from inside the Docker network.

The Lab 7 folder was not changed. The Dockerfiles in `9.Docker/` use `../7.ProximitySearchAPI` as their build context, so the code that was graded in Lab 7 is exactly the code in the images. Replaying the seven queries saved in the Lab 7 report through the containers gave byte-identical responses (only the start-up time field differs), and the 21 Lab 7 tests pass inside the api image (Section 3.5).

2 Docker design choices

Table 1: Design choices and the reason for each

Choice	Reason
Base images	
`python:3.12-slim` (api, loadtest)	Debian-based, so the manylinux wheels of numpy, uvloop and httptools install without compiling. Alpine uses musl and would build numpy from source. Slim is about 4× smaller than the full `python` image. Python 3.12 is the version Lab 7 was tested with.
`nginx:1.27-alpine` (web)	nginx needs nothing from glibc. The image is 76 MB and has `wget` for the healthcheck. It has an `envsubst` template step built in.

Choice	Reason
Two-stage api build	The builder stage creates a venv and installs the wheels. The runtime stage copies only `/opt/venv` and `app/`. Measured: 367 MB (79.7 MB compressed) against 422 MB (107.7 MB) for a naive one-stage Dockerfile (Figure 4).
Pinned tags	`nginx:1.27-alpine`, `python:3.12-slim`, and pinned versions in `requirements.txt`, so a rebuild next month gives the same stack.
Ports	
`8088:80` on web only	The single entry point. The api replicas publish nothing (`EXPOSE 8000` is documentation only), so they cannot be reached from the host and cannot clash on a host port when scaled. 8080 was already taken on this laptop by Docker Desktop, so the host port is the variable `WEB_PORT` (default 8088).
Networks	
`frontend` (bridge)	web and loadtest. It is the side that faces clients.
`backend` (bridge, `internal: true`)	web and api. "Internal" means no gateway: the api has no route to the internet or the host (shown in Figure 12). The API needs no outbound access, so it gets none.
Docker DNS	Containers find each other by service name (`api`, `web`). nginx re-resolves `api` every 10 s (`resolver 127.0.0.11 valid=10s`), so replicas added with `-scale` are used without restarting nginx.
Volumes	
Bind mount `../7.Proximity-SearchAPI/data :/data:ro`	The dataset (`locations.csv`, `link.txt`) is data, not code. Mounting it means a new road file needs a restart, not a rebuild. `:ro` because the API only reads it. The image's build context excludes `data/`.
Named volume `web-logs`	nginx writes a copy of its access log there. It survives `docker compose down` and is removed only by `down -v` (Figure 13).
Environment variables	
`PORT`, `WORKERS`	uvicorn port and worker processes, read by the `CMD`. `API_WORKERS` in `.env` sets them per run.
`LOCATIONS_CSV`, `LINKS_TXT`	Lab 7's loader already read these. They point it at `/data`.
`API_UPSTREAM`	Turned into the nginx config by `envsubst` at start-up, so the proxy target is not hard-coded in the image.
`API_REPLICAS`, `WEB_PORT`	Compose-level variables (`deploy.replicas`, port mapping) with defaults 1 and 8088. `.env.example` lists them.
`PYTHON-UNBUFFERED=1`	Python prints logs immediately, so `docker compose logs` shows them live.
Running safely	
Non-root user	The api runs as `app` (uid 10001). Its packages are owned by root, so the process cannot change its own code or libraries.
Healthchecks	api: Python `urllib` GET of `/health` (slim has no curl). web: `wget` of `/nginx-health`. `depends_on: condition: service_healthy` starts nginx only after the API has loaded the graph.
`restart: unless-stopped`	A crashed process is restarted by Docker. A manual `docker stop` is respected.
`exec` in `CMD`	uvicorn becomes PID 1, so `docker stop`'s SIGTERM reaches it and it shuts down cleanly, instead of `sh` ignoring the signal until the 10 s kill.
Per-Dockerfile `.dockerignore`	`api/Dockerfile.dockerignore` allows only `requirements.txt` and `app/`. The build context sent to Docker is 14 kB, not the 3 MB Lab 7 folder with its PDFs.

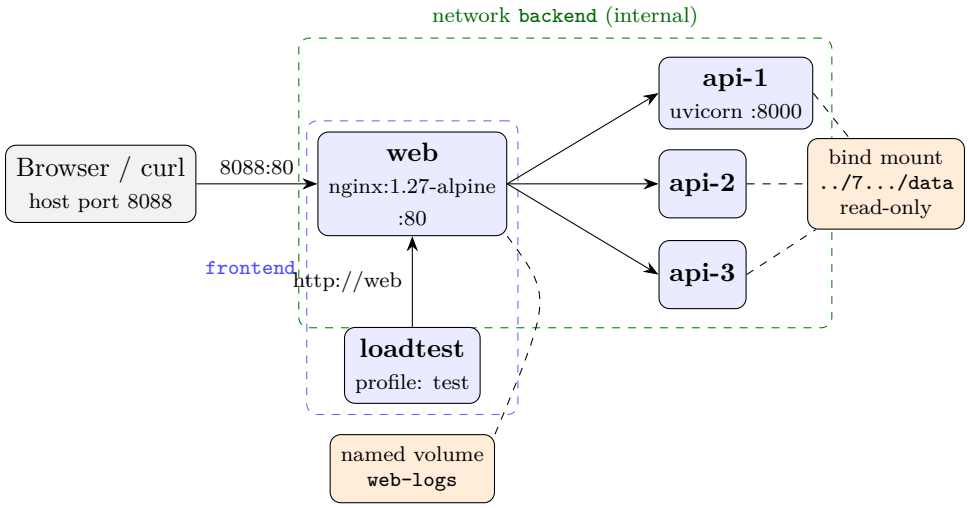


Figure 1: The stack. Only `web` has a host port. The api replicas sit on an internal network with no route out, and nginx spreads requests over them.

3 Steps followed

All steps were run on the personal laptop, not on the allotted lab systems. Every figure in this section is the real terminal output of that step, saved to `report/captures/*.txt` as it ran and rendered as an image. Figure 7 is a browser screenshot of the running stack.

3.1 Install

Docker Desktop for Mac (Apple silicon) was installed from docker.com. It bundles the engine, the `docker` CLI and the Compose plugin, and runs the engine in a small Linux VM, which is why the server reports `linux/arm64`. `hello-world` checks that the CLI can pull and run an image.

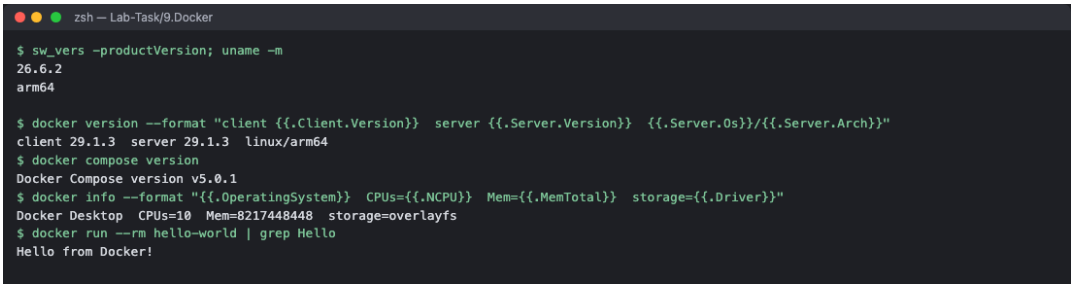


Figure 2: Install check: versions, the engine's VM, and `hello-world`.

3.2 Write the files

`api/Dockerfile`, `api/Dockerfile.dockerignore`	API image (two stages)
`web/Dockerfile`, `web/nginx.conf.template`	nginx image and its config
`loadtest/Dockerfile`	load-test client
`docker-compose.yml`, `.env.example`	the stack
`scripts/curl_checks.sh`, `scripts/render_terminal.py`	checks, and the figures in this report

The full files are in Appendix A.

3.3 Build

`docker compose build` builds every service that has a `build:` section. With `-no-cache` each step runs again; on this machine the three images took 16 s, mostly `pip install`. The build contexts

are 68–261 bytes per step because of the ignore files.

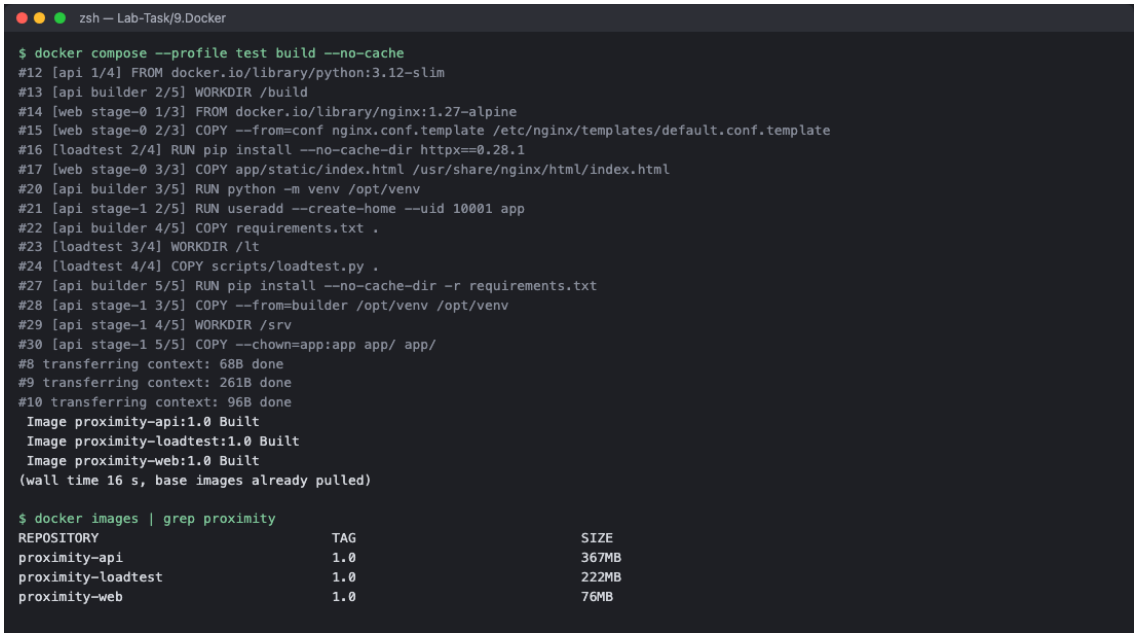


Figure 3: Build: the stages of each image. `[api builder]` and `[api stage-1]` are the two stages of the api image.



Figure 4: The two-stage api image against a one-stage version, and its layers. The runtime image is the slim base plus the 128 MB venv and the 53 kB of code.

3.4 Run

`docker compose up -d` creates the two networks, the volume, and the containers, in dependency order. It waits until all three api replicas are healthy before starting web.

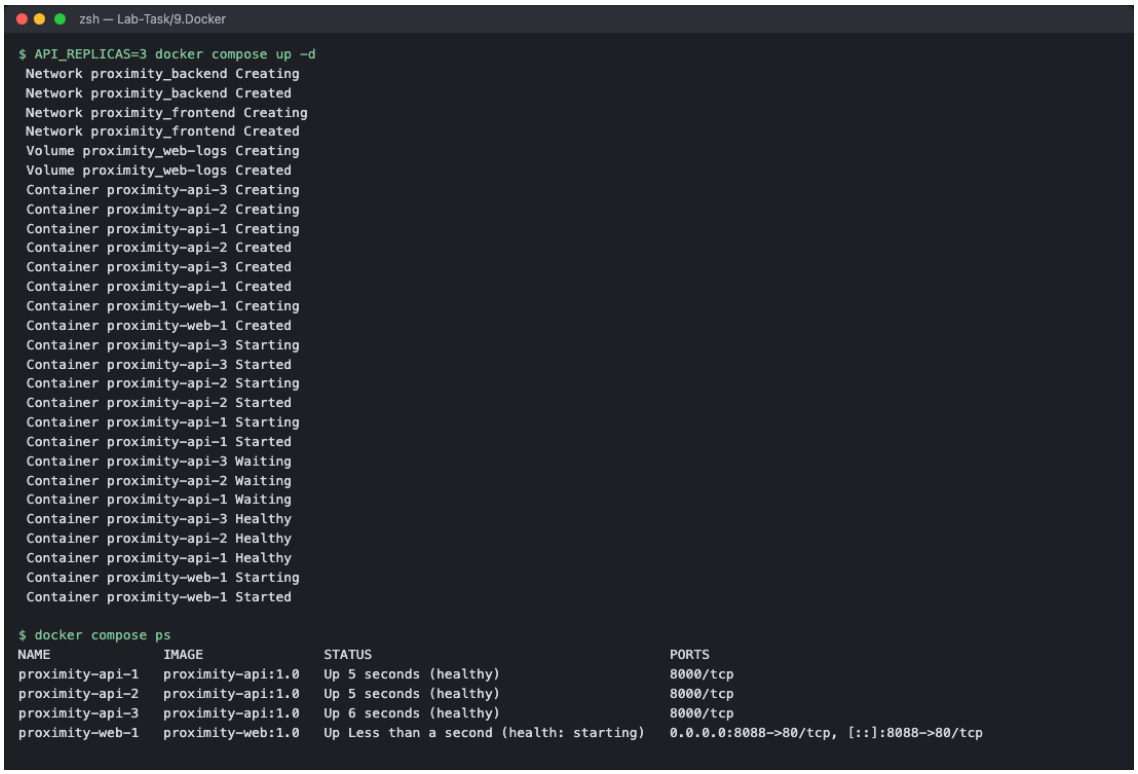


Figure 5: Run with three api replicas. Only `web` has a host port.

3.5 Verify



Figure 6: The seven queries from the Lab 7 report, now through nginx. Results and error messages match Lab 7 exactly, including `/search` without the trailing slash and the 400 errors.

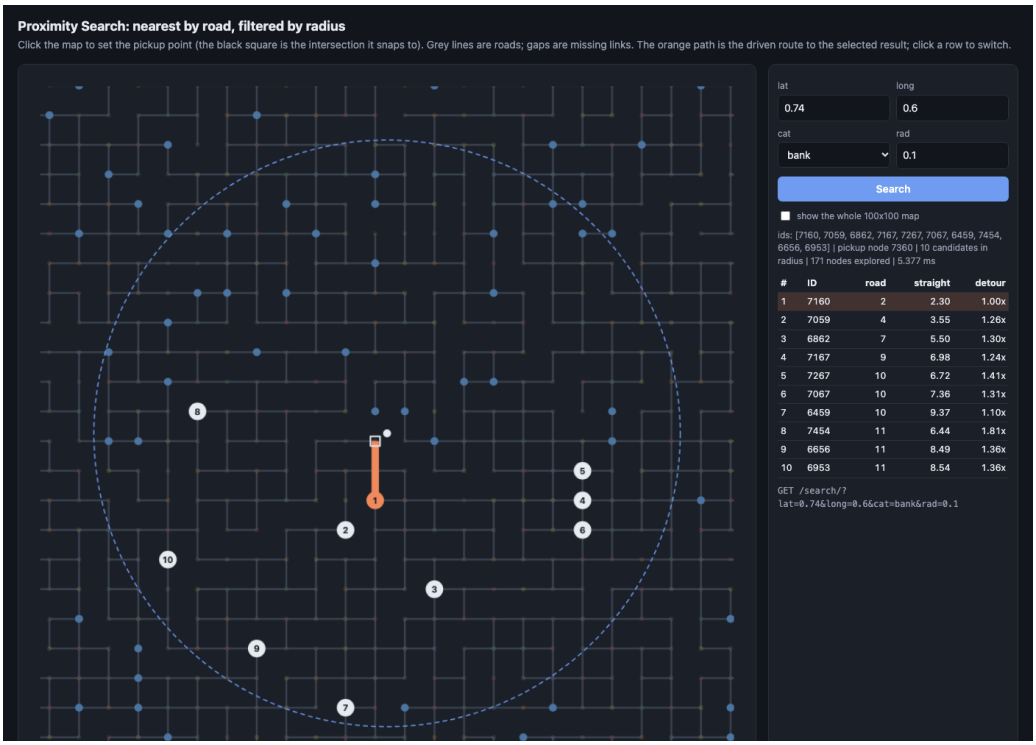


Figure 7: Browser screenshot of `http://localhost:8088/`. The page is served by nginx, and its `/network` and `/search/` calls go through nginx to an api replica.

Figure 8: FastAPI's `/docs` through the same proxy.



Figure 9: The 21 Lab 7 tests run inside the api image, with the Lab 7 folder mounted read-only. This tests the code against the exact Python and library versions that run in production.

3.6 Scale and load

nginx adds an `X-Upstream` header with the replica that answered. Nine requests reached all three replicas, with no nginx restart after scaling. The 5,000-request load test runs in its own container and reaches nginx at `http://web`.



Figure 10: Requests spread over the three replicas (IPs match `docker inspect`).

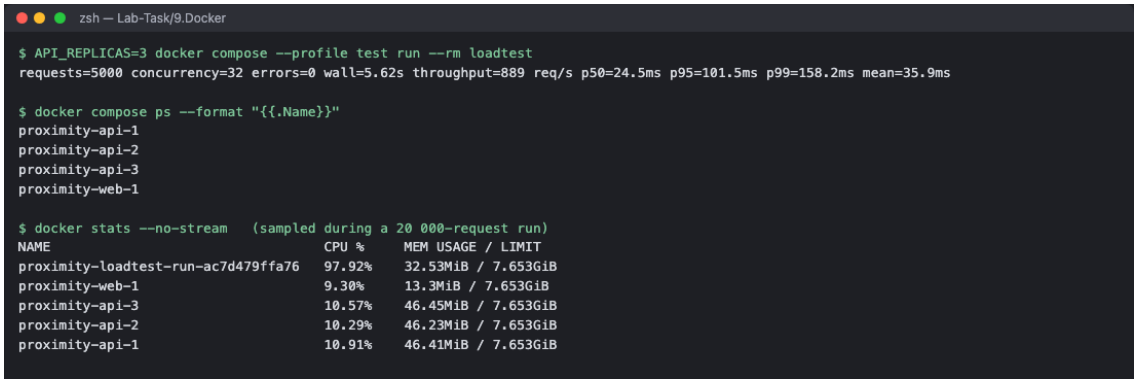


Figure 11: Load test, 5,000 requests, 32 concurrent: 0 errors, about 890 req/s, p50 25 ms. The three replicas stay up after `compose run`, which was the fix for error 2 below.

The load test is *client-bound*. During the run the load-test container uses 98 % of one CPU and each replica only about 10 %. Throughput was the same with one replica (884 req/s) as with three (889 req/s). So these numbers measure the Python load generator, not the API. A faster client (or several) would be needed to measure the API's limit. Lab 7 measured the search itself at 0.016 ms per query.

3.7 Networks, volumes, environment, restart

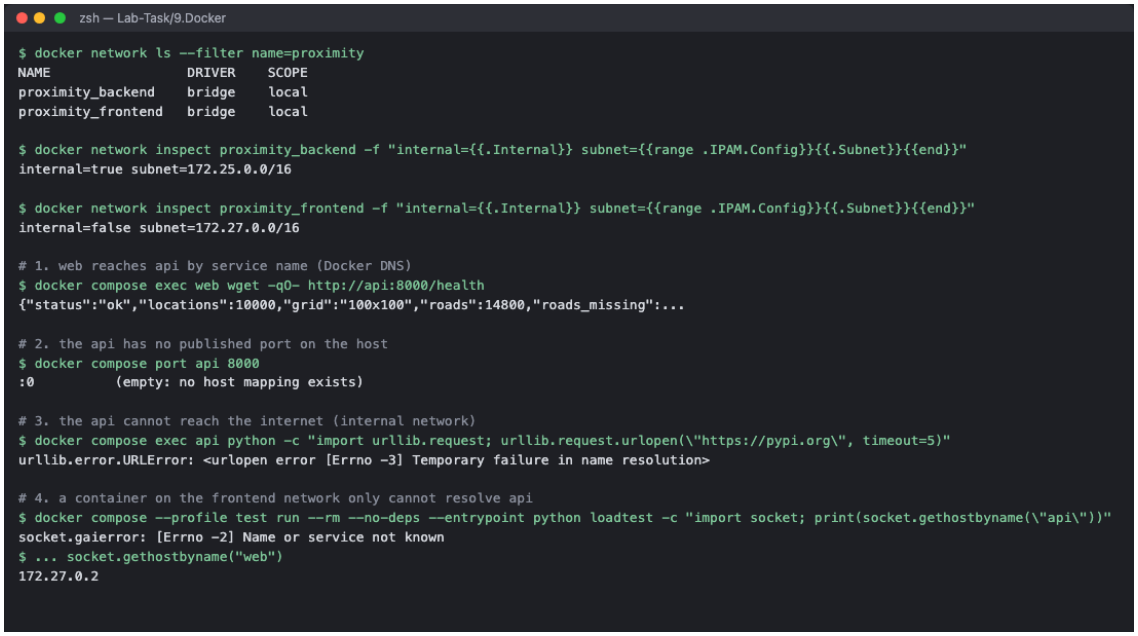


Figure 12: Network isolation. web reaches `api` by name; the api has no host port, cannot resolve anything outside, and cannot even be resolved from the frontend network.

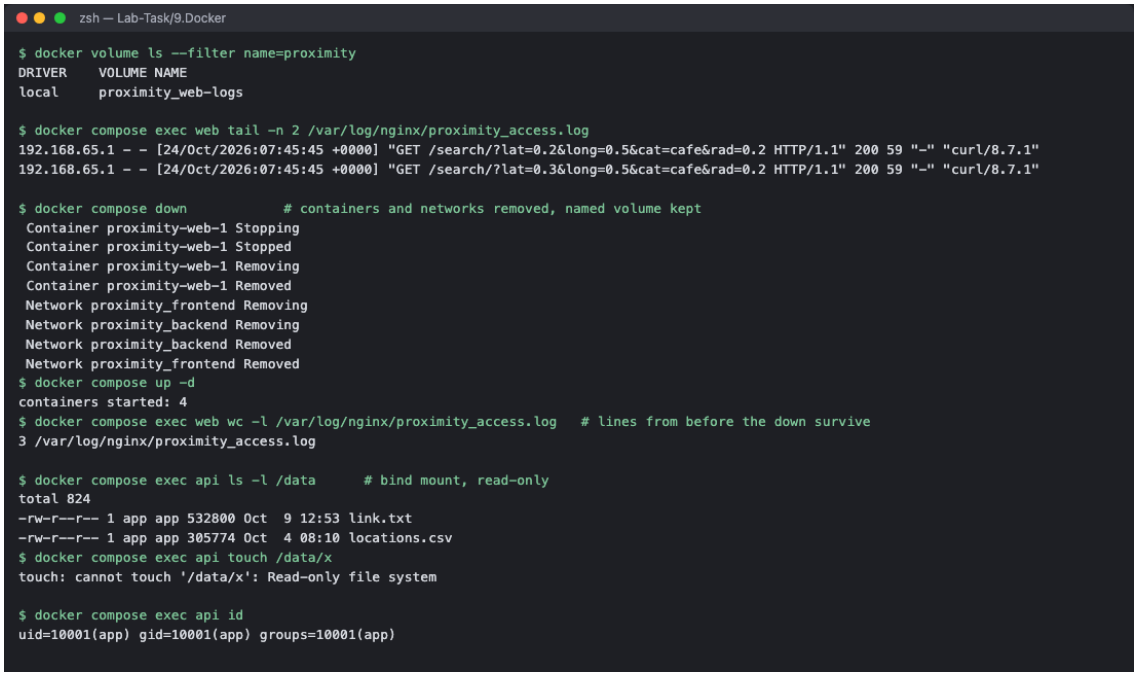


Figure 13: Volumes. The access log survives `compose down`/`up`. The dataset mount is read-only, and the process runs as uid 10001.

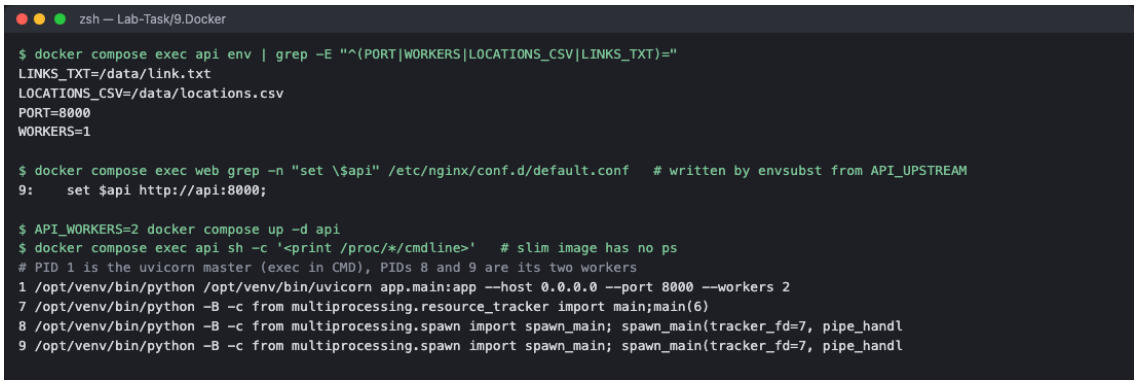


Figure 14: Environment variables inside the containers. With `API_WORKERS=2` uvicorn runs as PID 1 with two worker processes.

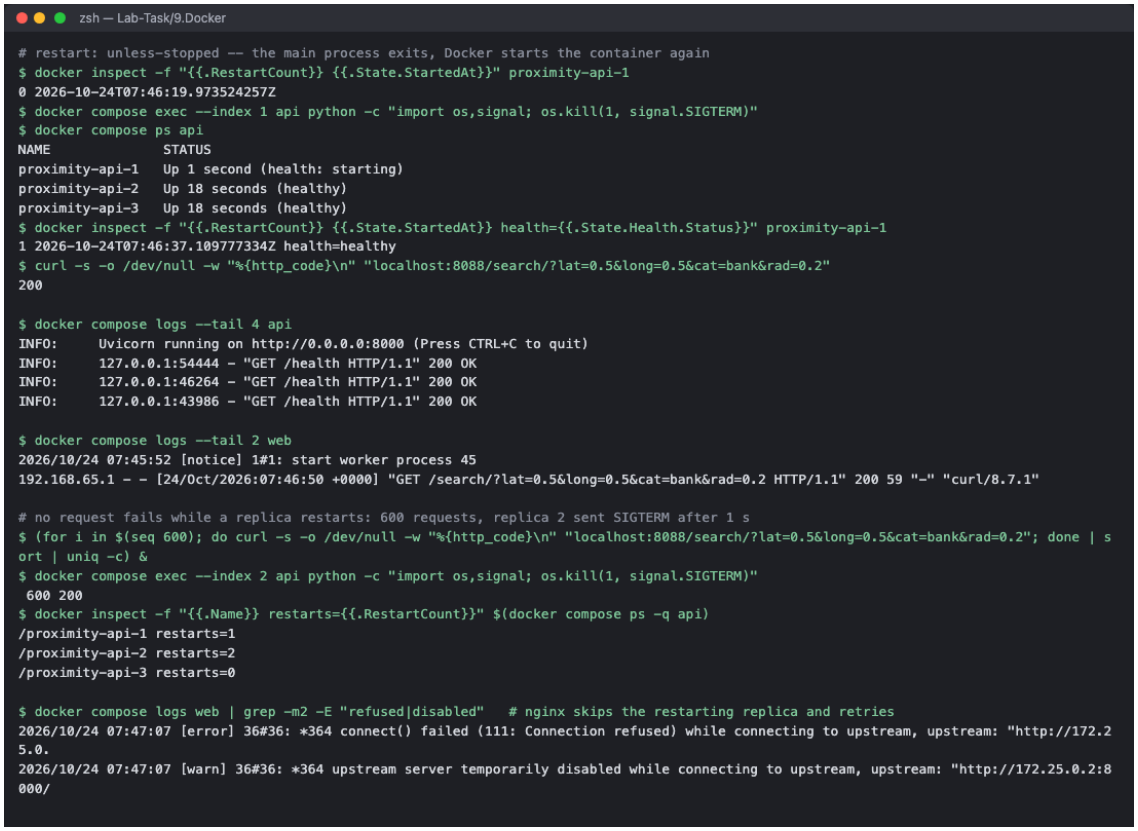


Figure 15: Self-healing. Sending SIGTERM to a replica's main process stops it, and Docker restarts it (`RestartCount` 0→1). In 600 requests sent while a replica restarted, all 600 returned 200, because nginx retried the other replicas.

3.8 Tear down

`docker compose down` removes containers and networks and keeps the `web-logs` volume. `docker compose down -v` removes the volume too. `-rmi local` would also delete the three built images.

4 Errors faced and how they were resolved

1. `pip install` in the api image: *Permission denied.* To run the Lab 7 tests inside the image, pytest has to be installed into it first. As the default user this failed:

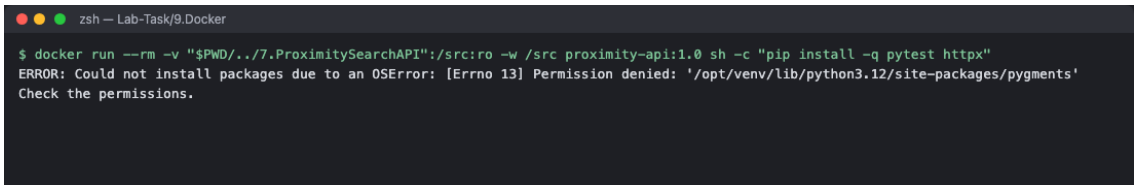


Figure 16: Error 1.

Cause: the image runs as the non-root user `app`, and `/opt/venv` belongs to root, which is the point of the non-root user. *Fix:* the image was left as it is. The one-off test container runs with `-u root`; the stack's own services never do. An alternative would be a separate `test` stage in the Dockerfile with the dev dependencies.

2. `docker compose run` removed two of three replicas. After `docker compose up -d --scale api=3`, running the load test with `docker compose run --rm loadtest` stopped and removed `api-4` and `api-5`:

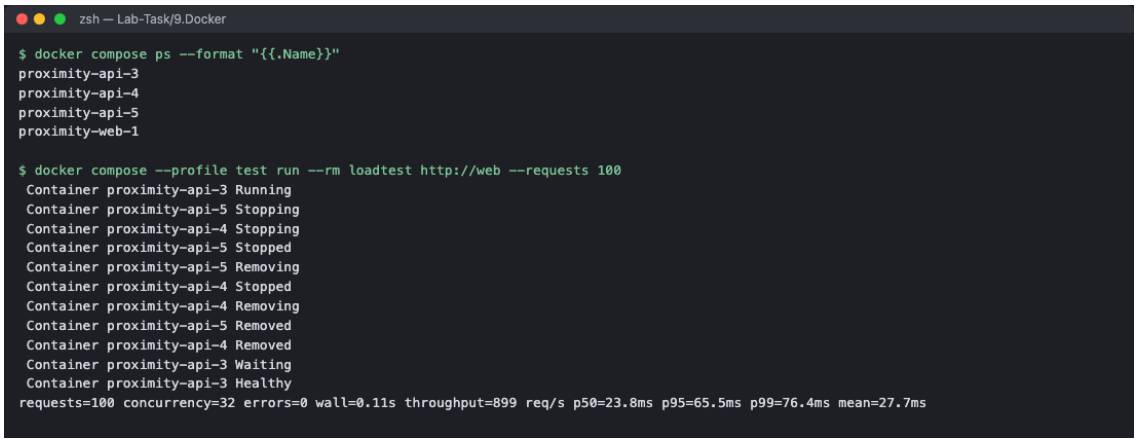


Figure 17: Error 2: `run` brings dependencies back to their configured replica count.

Cause: `run` first makes sure the service's dependencies (web, and through web, api) are running "as configured". The configuration said one api, and `-scale` on the earlier command line was not part of it. The first time this appeared it was hidden by `run -build`, which also rebuilt the api image. *Fix:* the number of replicas moved into the file, `deploy.replicas: ${API_REPLICAS:-1}`, so every command sees the same desired state (`API_REPLICAS=3 docker compose up -d`). The loadtest image is built once with `docker compose -profile test build loadtest`, then run without `-build`.

3. The log volume stored nothing. The `web-logs` volume was mounted at `/var/log/nginx`, but the directory only held two links:



Figure 18: Error 3: the official nginx image links its log files to stdout and stderr.

Cause: the official image does this on purpose, so that `docker logs` works. When a new named volume is mounted, Docker copies the image's directory into it, links included, so the log still

went to stdout and the volume was empty. *Fix:* a second `access_log` directive writes to the file `proximity_access.log` on the volume, while `access_log /dev/stdout` keeps `docker compose logs web` working (Figure 13).

4. Port 8080 already in use. Before choosing the host port, `lsof -iTCP:8080 -sTCP:LISTEN` showed Docker Desktop already listening there, and ports 3000, 3001, 8000 and 9090 belonged to another project's containers on the same laptop. Publishing on 8080 would have failed with "port is already allocated". *Fix:* the host port is the variable `WEB_PORT` with default 8088. A machine where 8088 is taken needs only `WEB_PORT=9000 docker compose up -d`.

5 Docker commands

5.1 Commands used, in order

```
docker version; docker compose version; docker info; docker run --rm hello-world
docker compose config -q # validate the compose file
docker compose --profile test build --no-cache # build api, web, loadtest
docker images | grep proximity; docker history proximity-api:1.0
API_REPLICAS=3 docker compose up -d # start web + 3 api replicas
docker compose ps
scripts/curl_checks.sh # the Lab 7 queries through nginx
docker run --rm -u root -v "$PWD/../7.ProximitySearchAPI":/src:ro -w /src \
    proximity-api:1.0 sh -c "pip install -q pytest httpx; python -m pytest -q"
docker compose up -d --scale api=3
docker inspect -f '{{.Name}} {{range $k,$v := .NetworkSettings.Networks}}{{$k}}={{$v.IPAddress}}
     {{end}}' $(docker compose ps -q)
docker compose --profile test build loadtest
docker compose --profile test run --rm loadtest
docker stats --no-stream
docker network ls; docker network inspect proximity_backend
docker compose exec web wget -qO- http://api:8000/health
docker compose port api 8000
docker compose exec api python -c "import urllib.request; urllib.request.urlopen('https://pypi.
    org')"
docker compose --profile test run --rm --no-deps --entrypoint python loadtest -c "..."
docker volume ls
docker compose exec web tail -n 2 /var/log/nginx/proximity_access.log
docker compose down; docker compose up -d
docker compose exec api env; docker compose exec api id
API_WORKERS=2 docker compose up -d api
docker compose exec --index 1 api python -c "import os,signal; os.kill(1, signal.SIGTERM)"
docker inspect -f '{{.RestartCount}} {{.State.Health.Status}}' proximity-api-1
docker compose logs --tail 4 api
docker build -t proximity-api:single -f Dockerfile.single . # size comparison only
docker rmi proximity-api:single
docker compose --profile test down -v
```

5.2 Unique commands and their purpose

Command	Purpose
`docker version` / `info`	client and engine versions; engine OS, CPUs, memory, storage driver
`docker run`	create and start a container from an image (`-rm` delete on exit, `-v` mount, `-u` user, `-w` working dir)
`docker build`	build an image from a Dockerfile and a context (`-t` tag, `-f` file)
`docker images`	list local images and their sizes
`docker history`	the layers of an image and the instruction that made each
`docker rmi`	delete an image
`docker inspect`	full JSON of a container, image, network or volume; `-f` picks fields with a Go template
`docker stats`	live CPU and memory per container (`-no-stream` one sample)
`docker network ls` / `inspect`	list networks; show subnet, `Internal`, attached containers
`docker volume ls`	list volumes
`docker compose config`	merge the file with `.env` and print it; `-q` only validates
`docker compose build`	build every service with a `build:` section (`-no-cache`, `-profile`)
`docker compose up -d`	create networks, volumes and containers, start in dependency order, detached; `-build` rebuilds first, `-scale svc=N` sets replicas
`docker compose ps`	the stack's containers, status, health and ports
`docker compose run -rm`	one-off container for a service (here the load test); `-no-deps`, `-entrypoint` override
`docker compose exec`	run a command in a running service container; `-index` picks a replica
`docker compose logs`	the services' stdout and stderr (`-tail`, `-f`)
`docker compose port`	which host port a container port is published on
`docker compose down`	stop and remove containers and networks; `-v` also volumes, `-rmi` images
`lsof -iTCP:8080 -sTCP:LISTEN`	(not Docker) which process already holds a host port

6 What I learnt

- **An image is the code plus its exact runtime.** The Lab 7 tests passing inside the image means the API works with that Python, numpy and uvicorn, not only with whatever was on my laptop. On the lab systems in Lab 7 I had to ship wheels by hand because the boxes had no internet. An image solves that problem in one artifact.
- **Layers and their order matter.** Copying `requirements.txt` and installing before copying `app/` means a code change rebuilds only the last 53 kB layer, not the 128 MB of packages. A multi-stage build leaves build-only files behind; here that saved 55 MB.
- **The build context is everything sent to the engine.** A per-Dockerfile ignore file cut it from the whole Lab 7 folder to 14 kB, and kept the dataset out of the image.
- **Networks are a security boundary I can declare.** `internal: true` and publishing only nginx means the API is unreachable except through the proxy, and the API itself cannot reach out. Docker's DNS replaces hard-coded IPs, which is what makes `-scale` work.
- **Compose is desired state, not a list of commands.** Error 2 showed this clearly: a replica count given only on the command line was undone by the next command. Anything that should persist belongs in the file.
- **Data survives only where a volume is.** Containers are disposable: `down` deletes their filesystems. Named volumes and bind mounts are the only persistent parts, and an image's own setup (error 3) can make a volume silently useless.

- **Healthchecks turn "started" into "ready".** `depends_on: service_healthy` and the healthcheck together make nginx wait until the graph is loaded. A restart policy plus several replicas behind nginx meant a crashed replica cost zero failed requests.
- **Measure what you think you are measuring.** `docker stats` showed the load generator, not the API, was the bottleneck. Without it, 890 req/s would have looked like the API's limit.

A Files

`docker-compose.yml`

```yaml
# Lab 9: the Lab 7 Proximity Search API as a containerised stack.
#
# docker compose up -d --build # web + api
# API_REPLICAS=3 docker compose up -d # three api replicas behind nginx
# docker compose --profile test run --rm loadtest
# docker compose down -v
name: proximity

x-lab7: &lab7 ../7.ProximitySearchAPI

services:
  api:
    build:
      context: *lab7
      dockerfile: ../9.Docker/api/Dockerfile
    image: proximity-api:1.0
    environment:
      PORT: 8000
      WORKERS: ${API_WORKERS:-1}
      LOCATIONS_CSV: /data/locations.csv
      LINKS_TXT: /data/link.txt
    volumes:
      # dataset is mounted, not baked in: a new CSV needs a restart, not a rebuild
      - ../7.ProximitySearchAPI/data:/data:ro
    networks: [backend]
    deploy:
      # kept in the file, not only on the command line: `compose run` resets
      # dependencies to this number (see report, errors section)
      replicas: ${API_REPLICAS:-1}
    restart: unless-stopped
    # no ports: the api is reachable only through web

  web:
    build:
      context: *lab7
      dockerfile: ../9.Docker/web/Dockerfile
      additional_contexts:
        conf: ./web
    image: proximity-web:1.0
    ports:
      - "${WEB_PORT:-8088}:80"
    environment:
      API_UPSTREAM: api:8000
    volumes:
      - web-logs:/var/log/nginx
    networks: [frontend, backend]
    depends_on:
      api:
        condition: service_healthy
    restart: unless-stopped

  loadtest:
```

```
    build:
      context: *lab7
      dockerfile: ../9.Docker/loadtest/Dockerfile
    image: proximity-loadtest:1.0
    networks: [frontend]
    depends_on:
      web:
        condition: service_healthy
    profiles: [test]

networks:
  frontend:
  backend:
    internal: true # no route to the outside world or the host

volumes:
  web-logs:
```

api/Dockerfile

```
# Proximity Search API (Lab 7), built from ../7.ProximitySearchAPI.
# Two stages: the builder installs the wheels into a venv, the runtime image
# copies only that venv and the code, so pip and build caches are left behind.

FROM python:3.12-slim AS builder
WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH=/opt/venv/bin:$PATH
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
ENV PATH=/opt/venv/bin:$PATH \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8000 \
    WORKERS=1
RUN useradd --create-home --uid 10001 app
COPY --from=builder /opt/venv /opt/venv
WORKDIR /srv
COPY --chown=app:app app/ app/
USER app
EXPOSE 8000
# slim has no curl, so the probe uses the Python standard library
HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import os,urllib.request; urllib.request.urlopen(f'http://127.0.0.1:{os.environ
      [\"PORT\"]}/health', timeout=2)" || exit 1
# exec so uvicorn is PID 1 and receives SIGTERM from `docker stop`
CMD ["sh", "-c", "exec uvicorn app.main:app --host 0.0.0.0 --port $PORT --workers $WORKERS"]
```

web/Dockerfile

```
# Front end: nginx serves the Lab 7 demo page and proxies the API routes to
# the api service. Build context is ../7.ProximitySearchAPI (for index.html).
FROM nginx:1.27-alpine
# files in templates/ go through envsubst at start-up (API_UPSTREAM)
COPY --from=conf nginx.conf.template /etc/nginx/templates/default.conf.template
COPY app/static/index.html /usr/share/nginx/html/index.html
ENV API_UPSTREAM=api:8000
EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD wget -q -O /dev/null http://127.0.0.1/nginx-health || exit 1
```

web/nginx.conf.template

```
server {
    listen 80;
    server_name _;

    # Docker's embedded DNS. Re-resolving every 10 s lets nginx pick up api
    # replicas added with `docker compose up --scale api=N` without a restart;
    # several A records for "api" are used round-robin.
    resolver 127.0.0.11 valid=10s ipv6=off;
    set $api http://${API_UPSTREAM};

    # the image links access.log to stdout, so a file is named explicitly to
    # keep a copy on the web-logs volume; stdout still feeds `docker compose logs`
    access_log /var/log/nginx/proximity_access.log combined;
    access_log /dev/stdout combined;

    root /usr/share/nginx/html;

    location = / {
        try_files /index.html =404;
    }

    location ~ ^/(search|network|health|docs|openapi\.json) {
        proxy_pass $api;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        # which api container answered, to show the round-robin
        add_header X-Upstream $upstream_addr always;
    }

    location = /nginx-health {
        access_log off;
        return 200 "ok\n";
    }
}
```

loadtest/Dockerfile

```
# Runs the Lab 7 load test from inside the compose network, against the web service.
FROM python:3.12-slim
RUN pip install --no-cache-dir httpx==0.28.1
WORKDIR /lt
COPY scripts/loadtest.py .
ENTRYPOINT ["python", "loadtest.py"]
CMD ["http://web", "--requests", "5000", "--concurrency", "32"]
```

api/Dockerfile.dockerignore

```
# Build context is ../7.ProximitySearchAPI; only requirements.txt and app/ are needed.
*
!requirements.txt
!app/
**/__pycache__
```